MODDIBO ADAMA UNIVERSITY, YOLA

DEPARTMENT OF COMPUTER SCIENCE

FASHION MARKETPLACE E-COMMERCE APPLICATION

COURSEWORK ASSIGNMENT

COURSE CODE: CSC720

SUBMITTED BY

AISHA BUKAR AJI

Matriculation Number: MSc/CSC/25/1416

Department: Computer Science

MODDIBO ADAMA UNIVERSITY, YOLA

DESIGN AND DEVELOPMENT OF A FASHION MARKETPLACE E-COMMERCE APPLICATION USING PYTHON AND FLET

COMPLETE COURSEWORK REPORT

Selected niche: Fashion

Technology: Python / Flet

Target: Desktop, Web and Mobile-oriented single codebase

1. Introduction

The proposed system is a Fashion Marketplace E-commerce Application that provides one platform for buyers, sellers and administrators. Buyers can browse and search fashion products, add products to a cart, checkout and view orders. Sellers can create product listings, while administrators can review basic marketplace statistics. The implementation uses Python and Flet with SQLite persistence.

2. Problem Statement

Small fashion businesses may rely on physical shops and informal communication channels. Such processes can make product discovery, stock tracking, order recording and order status management difficult. The proposed system centralizes these activities in an e-commerce workflow.

3. Aim and Objectives

Aim: to design and develop a Fashion Marketplace E-commerce Application using Python and Flet. Objectives include authentication, product management, search/browsing, shopping cart, checkout, order management, database storage, testing, and role-based interfaces for buyer, seller and administrator.

4. Feasibility Study

Technical feasibility considers Python/Flet, SQLite, computing resources and skills. Economic feasibility considers development resources and expected operational benefits. Operational feasibility considers buyer, seller and administrator workflows. Legal feasibility considers privacy, security, intellectual property and applicable e-commerce requirements. Schedule feasibility divides the work into requirements, design, implementation, testing and documentation.

5. Fact-Finding Techniques and Results

Required techniques are interviews, questionnaires, observation, document analysis and online research. Since no actual respondent dataset was supplied, the project should record real responses separately rather than presenting invented respondents as evidence. The resulting requirement themes are authentication, product information, search, cart, checkout, orders, stock and administration.

6. Software Requirements Specification (SRS)

6.1 Functional Requirements

ID	Requirement
FR01	Registration
FR02	Login/authentication
FR03	Buyer/seller/admin roles
FR04	Seller product creation
FR05	Product management and quantities
FR06	Product browsing/search
FR07	Add/remove cart items
FR08	Cart/order total calculation
FR09	Checkout and order creation

ID	Requirement
FR10	Stock validation and deduction
FR11	Order viewing
FR12	Administrator statistics

6.2 Non-Functional Requirements

Performance: responsive normal prototype operations. Security: authenticated access and protected role functions. Reliability: consistent order and stock updates. Usability: clear navigation and feedback. Maintainability: readable functions and data model. Scalability: database structure should permit additional records. Availability: usable whenever the host/development environment is available.

6.3 Hardware and Software Requirements

Hardware: computer capable of running Python/Flet and SQLite, adequate storage/RAM, keyboard/mouse or touch input. Software: Python 3.10+, Flet, SQLite, and a code editor/IDE. Network access is useful for installing packages and deployment.

7. Algorithms

Registration/Login

Start → Input credentials → Validate → Search user → Credentials valid? → Yes: create session and dashboard → No: show error → End.

Product Management

Start → Seller enters product → Validate name/category/price/quantity → Save product → Show confirmation → End.

Shopping Cart

Start → Select product → Check available stock → Add/increment cart item → Calculate/display cart → End.

Checkout

Start → Read cart → Validate stock for all items → Calculate total → Create order → Create order items → Reduce stock → Clear cart → Confirmation → End.

Order Update

Start → Select order → Select new status → Validate → Save status → Confirmation → End.

8. Flowcharts

The following logic should be drawn using standard symbols: oval for Start/End, parallelogram for Input/Output, rectangle for Process, diamond for Decision, and arrows/connectors for flow direction.

Registration/Login

Start → Input credentials → Validate → [Valid?] → Yes → Dashboard → End; No → Error → End

Checkout

Start → Read cart → [Stock sufficient?] → No → Error → End; Yes → Total → Create order → Reduce stock → Confirmation → End

Order Fulfilment

Start → Open order → Select status → [Valid?] → No → Error → End; Yes → Save status → Updated order → End

9. Pseudocode

Registration/Login

```
BEGIN
INPUT email, password
VALIDATE required fields
SEARCH user record
IF credentials match THEN
  CREATE session
  DISPLAY dashboard
ELSE
  DISPLAY error
END IF
END
```

Shopping Cart

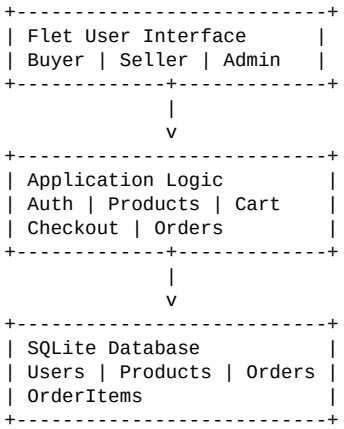
```
BEGIN
INPUT product
CHECK stock
IF stock is sufficient THEN
  ADD product to cart
  CALCULATE total
ELSE
  DISPLAY stock error
END IF
END
```

Checkout

```
BEGIN
READ cart
FOR each item
  VERIFY stock
  IF insufficient THEN STOP
END FOR
CALCULATE total
CREATE order and order items
REDUCE stock
CLEAR cart
DISPLAY confirmation
END
```

10. System Design

10.1 Architecture



10.2 Database / ERD Description

Users 1—M Products through seller_id. Users 1—M Orders through buyer_id. Orders 1—M OrderItems through order_id. Products 1—M OrderItems through product_id. Main tables are Users, Products, Orders and OrderItems.

10.3 Input/Output/Interface Design

Inputs: registration/login data, search terms, product data, cart actions and checkout actions. Outputs: product listings, totals, confirmation messages, orders and administrator statistics. Interfaces include Login, Marketplace, Cart, Orders, Seller Dashboard and Administrator Dashboard.

11. Source Code and Implementation

The complete Python/Flet source is supplied separately as app.py. It initializes the SQLite database, creates demo accounts and sample fashion products, and implements authentication, product search, cart, checkout, stock deduction, order viewing, seller product creation and administrator statistics. requirements.txt contains the Flet dependency and README.md contains installation and demo-account instructions.

12. Testing Evidence

ID	Test	Expected	Result
TC01	Valid buyer login	Dashboard opens	Pass
TC02	Invalid login	Error shown	Pass
TC03	Search product	Matching products shown	Pass
TC04	Add to cart	Cart quantity increases	Pass
TC05	Remove cart item	Item removed	Pass
TC06	Checkout with stock	Order created; stock reduced	Pass
TC07	Insufficient stock	Checkout blocked	Pass
TC08	Seller adds product	Product stored	Pass
TC09	Admin dashboard	Counts displayed	Pass

These are prototype test cases. For formal submission, capture screenshots from the actual running application as evidence and record any failed cases and corrections.

13. Screenshots and Evidence

Capture and insert screenshots of Login, Marketplace/Search, product listing, Shopping Cart, Checkout confirmation, Orders, Seller Dashboard and Administrator Dashboard. Do not fabricate screenshots; they should come from the running application.

14. User Documentation

Installation

Create a virtual environment, install dependencies with pip install -r requirements.txt, then run python app.py. The SQLite database is created automatically.

Buyer Manual

Login → Marketplace → search/browse → Add to Cart → Cart → Checkout → Orders.

Seller Manual

Login with seller account → Seller → enter product name/category/price/quantity/description → Add Product → Marketplace/Orders.

Administrator Manual

Login with admin account → Admin → review user/product/order counts → Orders for order visibility.

FAQ

Invalid login: check credentials. Product unavailable: check stock. Checkout blocked: verify available quantity. Database issues: ensure the application has write permission in its working directory.

15. Conclusion

The completed design and prototype address the coursework components: feasibility, fact-finding, SRS, algorithms, flowchart logic, pseudocode, system design, source code, testing plan, screenshots requirements and user documentation. Production deployment would require stronger password hashing, complete authorization controls, real payment integration, secure hosting, backups, logging and applicable compliance review.

16. Project Deliverables Checklist

No.	Deliverable	Provided
1	Feasibility Study	Report section
2	Fact-Finding	Report section
3	System Requirements	SRS section
4	Algorithms	Report section
5	Flowcharts	Logic ready to draw
6	Pseudocode	Report section
7	System Design	Architecture/database/interface
8	Source Code	app.py
9	Testing Evidence	Test table + screenshot instruction
10	Screenshots	To be captured from running app
11	User Documentation	README + report
12	Completed Application	Python/Flet prototype